

DISEÑO Y DESARROLLO DE SISTEMAS DE INFORMACIÓN

TEMA 4

OTROS MODELOS DE DATOS PARA SI



**UNIVERSIDAD
DE GRANADA**

**Antonio Cantillo Molina
Leandro Jorge Fernández Vega
Elena Abreu Fernández
Elena Gallardo Caro
Pablo Bolaños Martínez**

ÍNDICE

Información del grupo.....	3
1. Modelo Objeto - Relacional (Antonio Cantillo Molina).....	4
1.1 Instalación.....	4
1.2 Descripción del DDL Y DML utilizados.....	5
1.3 Sentencias empleadas.....	7
1.4 Discusión sobre la adecuación para el SI.....	17
2. Modelo NoSQL Documental (Elena Abreu Fernández).....	19
2.1 Instalación.....	19
2.2. DDL y DML utilizado.....	19
2.3. Sentencias empleadas y resultados.....	21
2.4. Discusión sobre la adecuación para el SI.....	26
3. Modelo NoSQL Clave-Valor (Elena Gallardo Caro).....	27
3.1 Descarga e instalación del SGBD.....	27
3.2. Descripción del DDL y DML utilizado.....	28
3.3. Sentencias empleadas y resultados.....	28
3.4. Discusión sobre la adecuación para el SI.....	31
4. Modelo NoSQL Orientado a Grafos (Pablo Bolaños Martínez).....	32
4.1. Descripción de la descarga e instalación del SGBD.....	32
4.2. Descripción del DDL y DML utilizado.....	32
4.3. Sentencias empleadas y Resultados.....	33
4.4. Discusión sobre la adecuación para el SI.....	36
5. Modelo Orientado a Objetos (Leandro Jorge Fernández Vega).....	37
5.1 Características.....	37
5.2 Instalación.....	37
5.3 Conexión a la base de datos y cierre.....	37
5.4. Descripción del DDL y DML utilizado.....	38
5.5. Discusión sobre la adecuación para el SI.....	41

Información del grupo

Grupo Pequeño (prácticas y seminarios)	A1
Grupo de trabajo	Uwuntu
Integrantes	Antonio Cantillo Molina Leandro Jorge Fernández Vega Elena Abreu Fernández Elena Gallardo Caro Pablo Bolaños Martínez

Alumno	Modelo	SGBD Elegido
Antonio Cantillo Molina	Objeto-Relacional	Oracle
Elena Gallardo Caro	NoSQL Clave-Valor	Redis
Elena Abreu Fernández	NoSQL Documental	MongoDB
Pablo Bolaños Martínez	NoSQL Grafos	Neo4j
Leandro Jorge Fernández Vega	Orientado a Objetos	ZODB

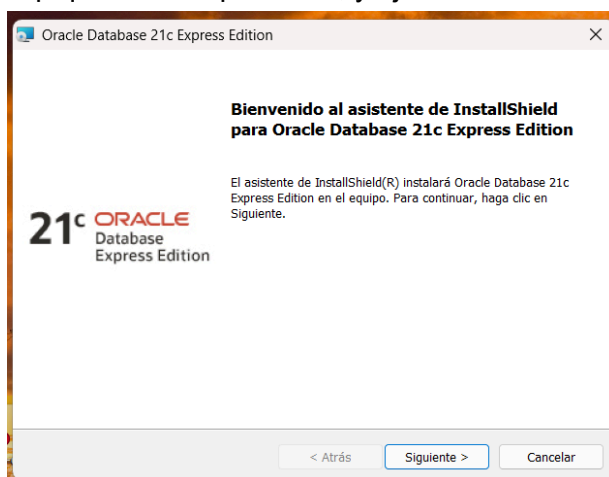
1. Modelo Objeto - Relacional (Antonio Cantillo Molina)

Para la implementación y desarrollo de este modelo, se ha decidido usar **Oracle Database Express Edition (XE)** como SGBD.

1.1 Instalación

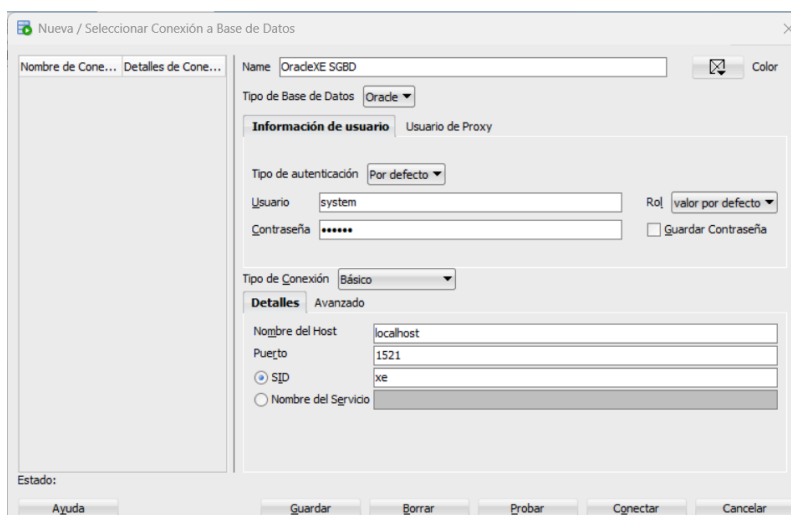
Comenzamos instalando la versión gratuita y ligera del SGBD Oracle: **Oracle Database Express Edition (XE)**.

Se nos descargará un .zip que descomprimos, y ejecutamos la instalación:

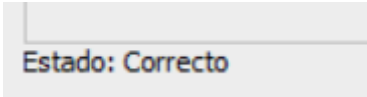


Una vez instalado, usaré Oracle SQL Developer (que ya tenía instalado) como interfaz gráfica para la ejecución de comandos sobre el SGBD.

A continuación (dentro de Oracle SQL Developer), nos conectamos al SGBD creando una nueva conexión. Para ello, usamos el botón “+” y rellenamos los datos asegurándonos de poner en usuario “system” y en **SID** “xe”:



y nos aseguramos de probar la conexión para ver que se ha efectuado todo correctamente:



Estado: Correcto

Por último, guardamos la conexión y nos conectamos a ella.

1.2 Descripción del DDL Y DML utilizados

Como se ha mencionado anteriormente, el SGBD utilizado es el de Oracle Database Express Edition (XE), que se trata de un SGBD objeto-relacional.

Las sentencias DDL y DML que se utilizan para este SGBD son:

- **CREATE OR REPLACE TYPE (nombre) AS OBJECT (...)** : Esta sentencia nos permite crear clases de objetos. Es decir, dentro de esta se especifican los atributos y métodos que un objeto de dicha clase posee.

Los atributos se crean así: **nombre TIPO**

Los métodos pueden ser de dos tipos:

- **MEMBER FUNCTION (nombre) RETURN (valor a devolver)**
- **MEMBER PROCEDURE (nombre)**

La primera de ellas permite definir una función, que realiza algún cálculo o proceso que es devuelto. Si por el contrario, no queremos devolver ningún valor, realizamos un procedimiento (**Segunda forma**).

Cabe recalcar que en la definición de la clase, sólo se definen las cabeceras de la funciones o métodos que queramos desarrollar.

A continuación, "OR REPLACE" simplemente permite sobrescribir. (Se incluyó para evitar problemas).

Por último, destacar que al final de toda esta sentencia se puede incluir **NOT FINAL** (es decir, **CREATE OR REPLACE TYPE nombre AS OBJECT (...) NOT FINAL;**). Esto nos permite introducir el concepto de herencia entre clases, que será utilizado en mi implementación.

Para las clases hija, es decir, aquellas que hereden de otra, la sintaxis de esta sentencia será la siguiente: **CREATE OR REPLACE TYPE (nombre) UNDER (nombre_clase_padre) (...)**;

- **CREATE OR REPLACE TYPE BODY (nombre) AS**
 (cabecera método) **IS**
 BEGIN
 (cuerpo)
 END;
 ...
END; :

Esta sentencia nos permite definir el cuerpo de las funciones definidas en la definición de la clase. Para cada función, bastará con poner la misma cabecera con la que se definió, seguido de un **IS** y definiendo su respectivo cuerpo entre **BEGIN** y **END**.

- **CREATE TABLE (nombre_tabla) OF (nombre_clase) (...); :** Esta sentencia equivale a la sentencia **CREATE** tradicional. Sin embargo, en caso de que se quiera que la tabla sólo esté compuesta por objetos de una clase, se añade **OF** para indicar que estará compuesta única y exclusivamente por objetos de tipo **(nombre_clase)**.

En caso de querer crear una tabla original o con mezcla de atributos normales y objetos, se hará de la siguiente manera:

```
CREATE TABLE (nombre_tabla) (  
  

    object nombre_clase,  

    fecha DATE,  

    cadena VARCHAR2(9)  

);
```

En caso de crear una tabla usando **OF**, NO se deben definir los atributos del objeto. ¿Por qué? La tabla no almacena los atributos para cada objeto, almacena INSTANCIAS de objetos de la clase especificada después de **OF**. La definición de dichos atributos ya se encuentra en la definición de la clase en sí.

Por último, destacar que en esta sentencia también se añadirán constraints, tanto como para definir las claves primarias, como para definir restricciones sobre atributos.

- **INSERT INTO (nombre_tabla) VALUES(constructor1(..., ...), constructor2(..., ...), ...):** Se trata de la sentencia clásica de inserción en tablas de la base de datos. Sin embargo, dado que ahora trabajamos con tipos y objetos, para la inserción de valores se necesita la creación de objetos de los tipos especificados en la definición. Por ello, ahora se utiliza el constructor de cada objeto que forme parte de la tabla **(nombre_tabla)**.
- **DELETE FROM (nombre_tabla) (alias) WHERE (condición):** Esta sentencia equivale a la sentencia **DELETE** tradicional. La única diferencia es la necesidad de en la condición, en caso de querer acceder a algún dato de algún objeto definido, necesitamos usar la notación de punto (Ejemplo: alias.nombre).

- **UPDATE (nombre_tabla) (alias) SET (Valores a cambiar) WHERE (condición):** De nuevo, esta sentencia es equivalente a la sentencia UPDATE tradicional. Sin embargo, de nuevo recalcar la necesidad de utilizar la notación de punto en caso de deseo de acceder a atributos de objetos.
- **SELECT (función), ... FROM (nombre_tabla) (alias) WHERE (condición):** Una vez más, parece la sentencia tradicional SELECT, pero esta vez hay un pequeño cambio. Ahora, en la parte en la que se especifican los atributos de las columnas que se quieren obtener, se puede incluir una función que hayamos definido previamente (solo se le llama poniendo su nombre).
- **DROP TABLE (nombre_tabla) / DROP TYPE (nombre_tipo):** La primera de ellas es la sentencia tradicional para eliminar tablas. La segunda sentencia se usa para eliminar clases o tipos de objetos.

1.3 Sentencias empleadas

Veamos ahora las sentencias empleadas para la implementación:

Comenzamos describiendo el pequeño esquema reducido de nuestro SI de un restaurante que se implementará:

- **Tabla Trabajadores:** Tabla que representa en el sistema a cada trabajador del restaurante. Contiene como campos el DNI (clave primaria) e información personal del trabajador.
- **Tabla Clientes:** Tabla que representa en el sistema a cada cliente del restaurante. Contiene como campos el DNI (clave primaria) e información personal del cliente.
- **Tabla reservas:** Tabla que representa en el sistema cada reserva del restaurante. Cada reserva es hecha por un solo cliente y cada una de ellas se le es asignada a un sólo trabajador. Contiene como campos: el identificador de la reserva (clave primaria), el DNI del cliente que realizó la reserva, el DNI del trabajador asignado a la reserva e información adicional de la reserva.

En primer lugar, creamos las clases. En nuestro caso, se ha creado una clase “persona”, que contempla los datos o atributos en común de trabajador y cliente. Esta clase será la clase padre de otras dos: trabajador y cliente. En adición, para intentar incluir una funcionalidad específica del modelo objeto-relacional, he incluido un método para la clase persona, que permite devolver en una misma línea el nombre, apellidos y DNI de la persona:

```
-- Creamos tipos para crear objetos que servirán para la definición de tablas:

-- Comenzamos creando un tipo "persona" para crear objetos
-- que guarden datos personales:
CREATE OR REPLACE TYPE persona AS OBJECT (
    nombre VARCHAR2(20),
    apellidos VARCHAR2(40),
    dni_nie VARCHAR2(9),
    email VARCHAR2(50),
    telefono VARCHAR2(15),
    MEMBER FUNCTION nombre_completo_dni RETURN VARCHAR2
) NOT FINAL;
/
```

La implementación del cuerpo de la función se ha hecho así:

```
CREATE OR REPLACE TYPE BODY persona AS
    MEMBER FUNCTION nombre_completo_dni RETURN VARCHAR2 IS
    BEGIN
        -- Concatenamos nombre, apellidos y dni:
        RETURN nombre || ' ' || apellidos || ' ' || dni_nie;
    END;
END;
/
```

“trabajador” y “cliente” son clases que heredarán de “persona”, y cada una de ellas, definirá adicionalmente los atributos específicos de los trabajadores y clientes, respectivamente:

```
-- Creamos el tipo de objeto "trabajador que hereda de "persona":
CREATE OR REPLACE TYPE trabajador UNDER persona (
    cuenta_bancaria VARCHAR2(24),
    cargo VARCHAR2(30),
    fecha_alta_trabajador DATE,
    estado_trabajador VARCHAR2(15)
);
/

-- Creamos el tipo de objeto "cliente" que hereda de "persona":
CREATE OR REPLACE TYPE cliente UNDER persona (
    fecha_alta_cliente DATE,
    contraseña VARCHAR2(20),
    estado_cliente VARCHAR2(15)
);
/
```


Por último, creamos la clase reserva, que incluye sus respectivos campos:

```
-- Creamos el tipo de objeto "reserva":
CREATE OR REPLACE TYPE reserva AS OBJECT (
    id_reserva VARCHAR2(10),
    dni_cliente VARCHAR2(9),
    dni_trabajador VARCHAR2(9),
    num_personas NUMBER,
    fecha_hora VARCHAR2(30),
    estado_reserva VARCHAR2(15)
);
/
```

A continuación, creamos tablas. Comenzamos creando las tablas de trabajador y cliente como tablas de objetos de tipo trabajador y de tipo cliente, respectivamente.

```
-- TABLA TRABAJADOR (Cada fila es un objeto de tipo trabajador)
CREATE TABLE TABLA_TRABAJADOR OF trabajador (
    CONSTRAINT pk_trabajador PRIMARY KEY (dni_nie),
    CONSTRAINT check_dni_trabajador CHECK (REGEXP_LIKE(dni_nie, '^([0-9]{8}[A-Z]$)'))
);
/

-- TABLA CLIENTE (Cada fila es un objeto de tipo cliente)
CREATE TABLE TABLA_CLIENTE OF cliente (
    CONSTRAINT pk_cliente PRIMARY KEY (dni_nie),
    CONSTRAINT check_dni_cliente CHECK (REGEXP_LIKE(dni_nie, '^([0-9]{8}[A-Z]$)'))
);
/
```

Como se ha mencionado anteriormente, sobre estas sentencias se define qué atributo del objeto se considerará clave primaria. Adicionalmente, consideramos la definición de “constraints” o restricciones adicionales: en este caso, se comprueba que las claves primarias (DNI/NIE) tiene formato correcto a la hora de introducir nuevas tuplas.

Y posteriormente, creamos la tabla de reservas:

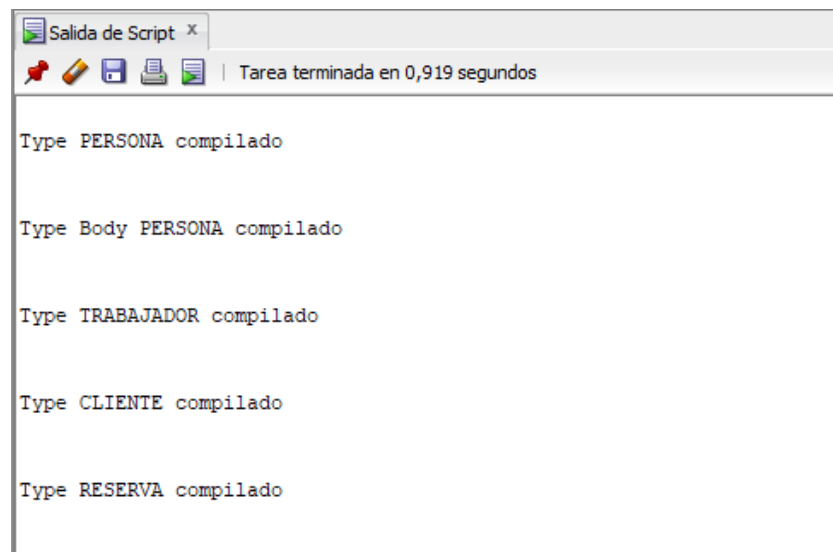
```
-- TABLA RESERVAS (Cada fila es un objeto de tipo reserva)
CREATE TABLE TABLA_RESERVA OF reserva (
    estado_reserva DEFAULT 'ACTIVO',
    CONSTRAINT pk_reserva PRIMARY KEY (id_reserva),
    CONSTRAINT fk_res_cliente FOREIGN KEY (dni_cliente) REFERENCES TABLA_CLIENTE(dni_nie),
    CONSTRAINT fk_res_trabajador FOREIGN KEY (dni_trabajador) REFERENCES TABLA_TRABAJADOR(dni_nie),

    CONSTRAINT check_id_reserva CHECK (REGEXP_LIKE(id_reserva, '^([0-9]{10}$)'))
);
/
```

De nuevo, se define la clave primaria "id_reserva" y se define una restricción para comprobar que el identificador de reserva cumple una forma. Muy importante destacar que dado que esta tabla guarda tuplas con dos atributos que son claves externas que referencian a las claves primarias de las tablas de trabajador y cliente, entonces hay que definirlas como tal (como claves externas).

Al definir el DNI/NIE del trabajador y del cliente como claves externas, saltará error si tratamos de añadir una reserva con un DNI/NIE (ya sea de trabajador o cliente) no existente en la base de datos.

Al ejecutar todo esto, claramente vemos que se crean:



```
Salida de Script x
Tarea terminada en 0,919 segundos

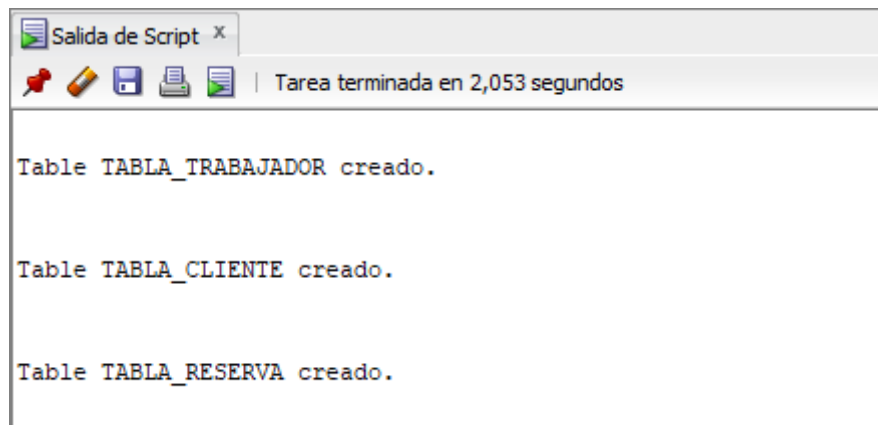
Type PERSONA compilado

Type Body PERSONA compilado

Type TRABAJADOR compilado

Type CLIENTE compilado

Type RESERVA compilado
```



```
Salida de Script x
Tarea terminada en 2,053 segundos

Table TABLA_TRABAJADOR creado.

Table TABLA_CLIENTE creado.

Table TABLA_RESERVA creado.
```

Ahora, procedemos a realizar inserciones. Recordemos que para añadir a estas tablas, hemos de utilizar los constructores propios de los tipos de objeto que incluirán. En caso contrario de no utilizarse, Oracle invocará internamente al constructor del objeto. En nuestro caso, se hace uso de los constructores explícitamente para menor confusión y para ser estricto con el modelo de objetos.

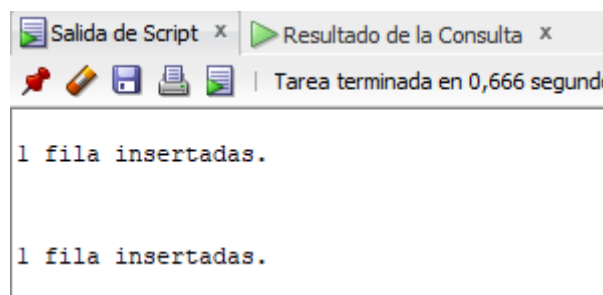
Como se ve a continuación, se utiliza dentro de la sentencia **INSERT** el constructor correspondiente al tipo de objetos que contiene la tabla **TABLA_TRABAJADOR**, que es **trabajador**:

```
-- Insertar un trabajador usando constructor de trabajador:
INSERT INTO TABLA_TRABAJADOR VALUES (
    trabajador('Antonio', 'Cantillo', '12345678A', 'antonio@correo.com',
        '123456789', 'ES910000111122223333', 'Camarero', SYSDATE, 'ACTIVO'));

INSERT INTO TABLA_TRABAJADOR VALUES (
    trabajador('Elena', 'Abreu', '12121212A', 'elena@correo.com',
        '123456789', 'ES910000111122223333', 'Cocinero', SYSDATE, 'ACTIVO'));

SELECT obj.* FROM TABLA_TRABAJADOR obj;
```

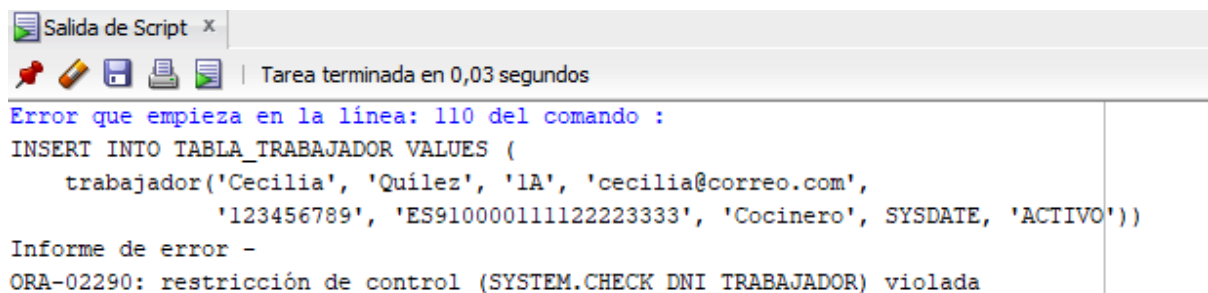
y vemos que resulta todo bien:



Salida de Script x Resultado de la Consulta x									
SQL Todas las Filas Recuperadas: 2 en 0,008 segundos									
NOMBRE	APELLIDOS	DNI_NIE	EMAIL	TELEFONO	CUENTA_BANCARIA	CARGO	FECHA_ALTA_TRABAJADOR	ESTADO_TRABAJADOR	
1 Antonio	Cantillo	12345678A	antonio@correo.com	123456789	ES910000111122223333	Camarero	01/01/26	INACTIVO	
2 Elena	Abreu	12121212A	elena@correo.com	123456789	ES910000111122223333	Cocinero	01/01/26	ACTIVO	

Comprobamos también que la introducción de un DNI/NIE con formato erróneo provoca error:

```
-- ERROR
INSERT INTO TABLA_TRABAJADOR VALUES (
    trabajador('Cecilia', 'Quílez', '1A', 'cecilia@correo.com',
        '123456789', 'ES910000111122223333', 'Cocinero', SYSDATE, 'ACTIVO'));
```



A continuación, realizamos inserciones para la tabla de clientes y reservas, usando sus respectivos constructores:

CLIENTE:

```
-- Insertar un cliente usando constructor de cliente:
INSERT INTO TABLA_CLIENTE VALUES (
    cliente('Leandro', 'Fernández', '12345678B', 'leandro@correo.com',
        '123456789', SYSDATE, 'secretal23', 'ACTIVO'));

INSERT INTO TABLA_CLIENTE VALUES (
    cliente('Pablo', 'Bolaños', '33333333C', 'pablo@correo.com',
        '123456789', SYSDATE, 'secretal23', 'ACTIVO'));

SELECT obj.* FROM TABLA_CLIENTE obj;
```

Salida de Script x

Resultado de la Consulta x



SQL | Todas las Filas Recuperadas: 2 en 0,004 segundos

	NOMBRE	APELLIDOS	DNI_NIE	EMAIL	TELEFONO	FECHA_ALTA_CLIENTE	CONTRASEÑA	ESTADO_CLIENTE
1	Leandro	Fernández	12345678B	leandro@correo.com	123456789	01/01/26	secretal23	ACTIVO
2	Pablo	Bolaños	33333333C	pablo@correo.com	123456789	01/01/26	secretal23	ACTIVO

RESERVA:

```
-- Insertar reservas usando constructor de reserva:
INSERT INTO TABLA_RESERVA VALUES (
    reserva('0000000001', '12345678B', '12345678A', 4, '10-10-2025 21:00', 'PENDIENTE')
);

INSERT INTO TABLA_RESERVA VALUES (
    reserva('0000000002', '33333333C', '12121212A', 4, '10-10-2025 21:00', 'CANCELADA')
);

SELECT obj.* FROM TABLA_RESERVA obj;
```

Salida de Script x Resultado de la Consulta 1 x

 SQL | Todas las Filas Recuperadas: 2 en 0,004 segundos

	ID_RESERVA	DNI_CLIENTE	DNI_TRABAJADOR	NUM_PERSONAS	FECHA_HORA	ESTADO_RESERVA
1	0000000001	12345678B	12345678A	4	10-10-2025 21:00	PENDIENTE
2	0000000002	33333333C	12121212A	4	10-10-2025 21:00	CANCELADA

Comprobamos además, que si trato de crear una reserva con DNI no existente (por ejemplo, de cliente), salta error:

```
Salida de Script x
Tarea terminada en 0,029 segundos

Error que empieza en la línea: 132 del comando :
INSERT INTO TABLA_RESERVA VALUES (
  reserva('00000000003', '99999999A', '12345678A', 4, '10-10-2025 21:00', 'PENDIENTE')
)
Informe de error -
ORA-02291: restricción de integridad (SYSTEM.FK_RES_CLIENTE) violada - clave principal no encontrada
```

A continuación, realizamos borrados. Como ya sabemos, si queremos filtrar, necesitamos acceder a los atributos con notación de punto:

```
-- Borrar un cliente:
DELETE FROM TABLA_CLIENTE obj WHERE obj.dni_nie = '12345678B';

SELECT obj.* FROM TABLA_CLIENTE obj;
```

Se realiza el borrado de un cliente. El estado de la tabla previo a realizar el borrado es:

	NOMBRE	APELLIDOS	DNI_NIE	EMAIL	TELEFONO	FECHA_ALTA_CLIENTE	CONTRASEÑA	ESTADO_CLIENTE
1	Leandro	Fernández	12345678B	leandro@correo.com	123456789	01/01/26	secretal23	ACTIVO
2	Pablo	Bolaños	33333333C	pablo@correo.com	123456789	01/01/26	secretal23	ACTIVO

Tras realizar el borrado, obtenemos el siguiente error. Este error se debe a que existe una reserva donde el DNI del cliente asociado a ella es el del cliente que queremos eliminar.

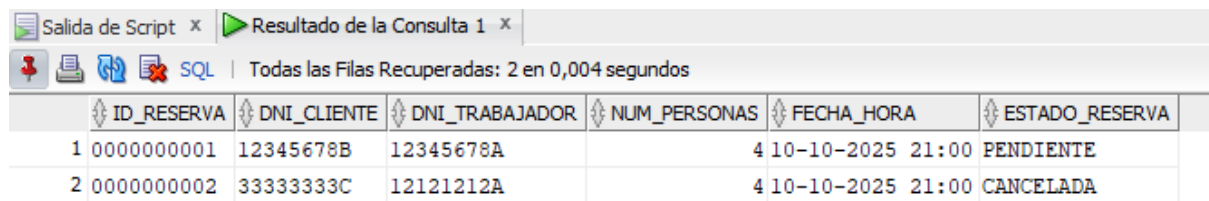
```
Error que empieza en la línea: 145 del comando :
DELETE FROM TABLA_CLIENTE obj WHERE obj.dni_nie = '12345678B'
Informe de error -
ORA-02292: restricción de integridad (SYSTEM.FK_RES_CLIENTE) violada - registro secundario encontrado
```

Sin embargo, si realizamos un borrado de reserva, vemos que se efectúa sin problemas:

```
-- Borrar reservas que hayan sido canceladas:
DELETE FROM TABLA_RESERVA obj WHERE obj.estado_reserva = 'CANCELADA';

SELECT obj.* FROM TABLA_RESERVA obj;
```

Estado de la tabla previo al borrado:

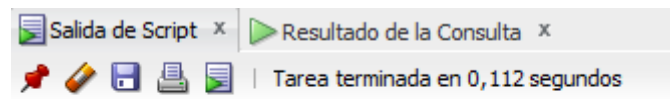


Salida de Script x Resultado de la Consulta 1 x

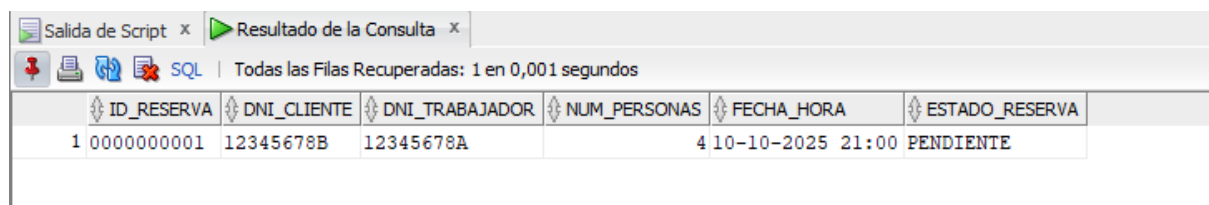
Todas las Filas Recuperadas: 2 en 0,004 segundos

	ID_RESERVA	DNI_CLIENTE	DNI_TRABAJADOR	NUM_PERSONAS	FECHA_HORA	ESTADO_RESERVA
1	0000000001	12345678B	12345678A	4	10-10-2025 21:00	PENDIENTE
2	0000000002	33333333C	12121212A	4	10-10-2025 21:00	CANCELADA

Tras el borrado:



1 fila eliminado



Salida de Script x Resultado de la Consulta x

Todas las Filas Recuperadas: 1 en 0,001 segundos

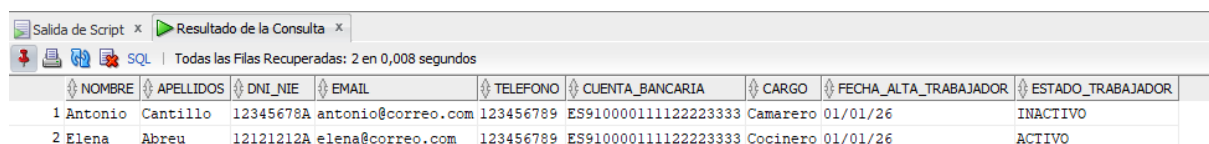
	ID_RESERVA	DNI_CLIENTE	DNI_TRABAJADOR	NUM_PERSONAS	FECHA_HORA	ESTADO_RESERVA
1	0000000001	12345678B	12345678A	4	10-10-2025 21:00	PENDIENTE

A continuación, vamos a realizar la modificación de atributos.

Comenzamos haciendo:

```
-- Cambiar cargo de un trabajador:  
UPDATE TABLA_TRABAJADOR obj SET obj.cargo = 'Jefe de Cocina' WHERE obj.dni_nie = '12345678A';  
  
SELECT obj.* FROM TABLA_CLIENTE obj;
```

Estado de la tabla previa a la modificación:

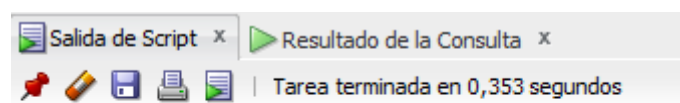


Salida de Script x Resultado de la Consulta x

Todas las Filas Recuperadas: 2 en 0,008 segundos

	NOMBRE	APELLIDOS	DNI_NIE	EMAIL	TELEFONO	CUENTA_BANCARIA	CARGO	FECHA_ALTA_TRABAJADOR	ESTADO_TRABAJADOR
1	Antonio	Cantillo	12345678A	antonio@correo.com	123456789	ES910000111122223333	Camarero	01/01/26	INACTIVO
2	Elena	Abreu	12121212A	elena@correo.com	123456789	ES910000111122223333	Cocinero	01/01/26	ACTIVO

Tras la modificación:



1 fila actualizadas.

Salida de Script x Resultado de la Consulta x

SQL | Todas las Filas Recuperadas: 2 en 0,001 segundos

NOMBRE	APELLIDOS	DNI_NIE	EMAIL	TELEFONO	CUENTA_BANCARIA	CARGO	FECHA_ALTA_TRABAJADOR	ESTADO_TRABAJADOR
1 Antonio	Cantillo	12345678A	antonio@correo.com	123456789	ES910000111122223333	Jefe de Cocina	01/01/26	INACTIVO
2 Elena	Abreu	12121212A	elena@correo.com	123456789	ES910000111122223333	Cocinero	01/01/26	ACTIVO

Vemos que el cambio se ha hecho efectivo, pues se ha cambiado el cargo a “Jefe de Cocina”.

O si hacemos esta otra modificación:

```
-- Cambiar estado reserva de 'PENDIENTE' a 'CONFIRMADA':
UPDATE TABLA_RESERVA obj SET obj.estado_reserva = 'CONFIRMADA' WHERE obj.id_reserva = '0000000001';

SELECT obj.* FROM TABLA_RESERVA obj;
```

Estado de la tabla previa a la modificación:

Salida de Script x

Resultado de la Consulta x



SQL

Todas las Filas Recuperadas: 1 en 0,001 segundos

ID_RESERVA	DNI_CLIENTE	DNI_TRABAJADOR	NUM_PERSONAS	FECHA_HORA	ESTADO_RESERVA
1 0000000001	12345678B	12345678A	4	10-10-2025 21:00	PENDIENTE

Tras la modificación:

Salida de Script x Resultado de la Consulta x
Tarea terminada en 0,35 segundos
1 fila actualizadas.

Salida de Script x

Resultado de la Consulta x



SQL

Todas las Filas Recuperadas: 1 en 0,002 segundos

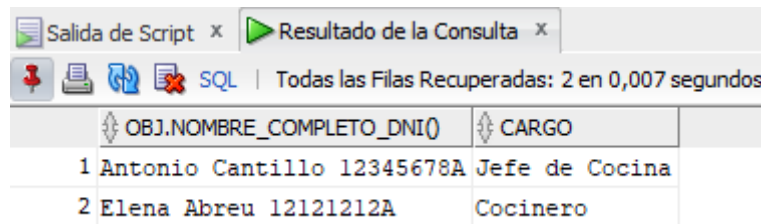
ID_RESERVA	DNI_CLIENTE	DNI_TRABAJADOR	NUM_PERSONAS	FECHA_HORA	ESTADO_RESERVA
1 0000000001	12345678B	12345678A	4	10-10-2025 21:00	CONFIRMADA

Por último, obtendremos valores de atributos de las tablas usando la sentencia SELECT:

Para comprobar el funcionamiento de la función “nombre_completo_dni” y obtener el cargo de un trabajador:

```
-- Obtener Nombre, Apellidos, DNI y cargo de todos los trabajadores:
SELECT obj.nombre_completo_dni(), obj.cargo FROM TABLA_TRABAJADOR obj;
```

Cuya salida muestra que todo funciona correctamente:



Salida de Script x Resultado de la Consulta x

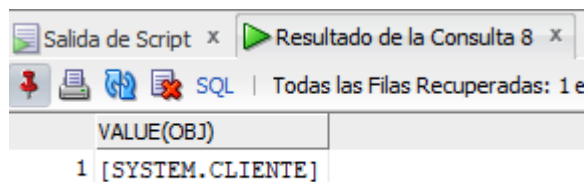
Todas las Filas Recuperadas: 2 en 0,007 segundos

	OBJ.NOMBRE_COMPLETO_DNI()	CARGO
1	Antonio Cantillo 12345678A	Jefe de Cocina
2	Elena Abreu 12121212A	Cocinero

También podemos obtener todo el objeto en sí que guarda una tabla:

```
-- Obtener todo un objeto cliente:  
SELECT VALUE(obj) FROM TABLA_CLIENTE obj WHERE obj.dni_nie = '12345678B';
```

En ese caso, la salida es la siguiente:

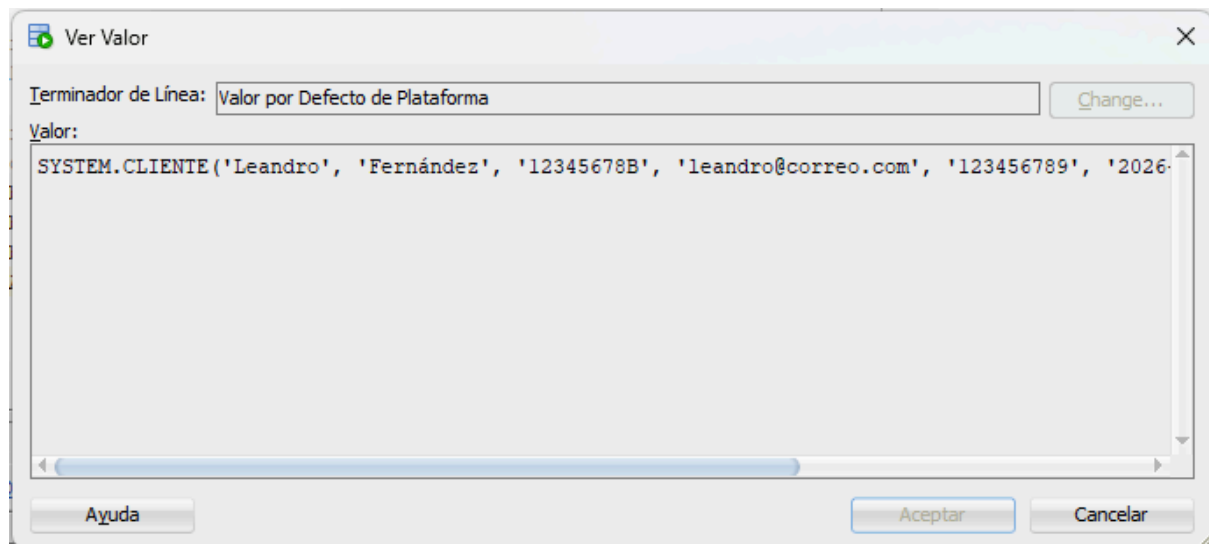


Salida de Script x Resultado de la Consulta 8 x

Todas las Filas Recuperadas: 1 en 0,007 segundos

	VALUE(OBJ)
1	[SYSTEM.CLIENTE]

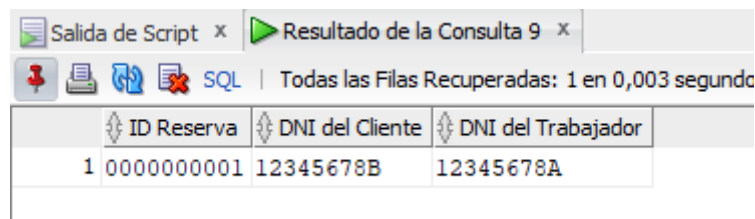
Podemos ver que se guarda un objeto, al cual podemos acceder haciendo clic sobre él obteniendo así su información. Esto nos hace ver la mezcla Objeto-Relacional de este modelo:



O ejecutando esta otra sentencia:

```
-- Obtener identificador de reserva, dni del cliente y trabajador
-- que hicieron cada reserva:
SELECT  obj.id_reserva AS "ID Reserva",
        obj.dni_cliente AS "DNI del Cliente",
        obj.dni_trabajador AS "DNI del Trabajador"
FROM TABLA_RESERVA obj;
```

Obtenemos:



The screenshot shows a database query result window with the title 'Resultado de la Consulta 9'. The window displays a table with three columns: 'ID Reserva', 'DNI del Cliente', and 'DNI del Trabajador'. The first row of data shows the values '1', '0000000001', and '12345678B' for the first two columns, and '12345678A' for the third column. The status bar at the bottom indicates 'Todas las Filas Recuperadas: 1 en 0,003 segundos'.

ID Reserva	DNI del Cliente	DNI del Trabajador
1	0000000001	12345678B

1.4 Discusión sobre la adecuación para el SI

El uso de un modelo Objeto-Relacional resulta muy adecuado para la implementación del sistema de información de un restaurante. Esto es debido a varias razones:

- En primer lugar, el hecho de considerar objetos es algo ventajoso. Ventajoso en el sentido de que designar clases de objetos, nos permite definir para cada objeto funcionalidades asociadas a él. Es decir, los objetos saben hacer cosas pues relacionamos a ellos métodos o funciones. Esto nos permite, por tanto, desarrollar funcionalidades o requisitos del SI en forma de métodos para cada objeto.
- El uso de referencias para conectar objetos (como las claves externas vista en la tabla implementada "reserva"), nos permite conectar objetos entre sí de manera simple. Por tanto, esto acelera las consultas.
- El concepto de herencia de objetos visto anteriormente, resulta extremadamente útil cuando consideramos objetos en nuestro SI. Por ejemplo, si en algún momento, deseamos añadir al SI un nuevo tipo de persona (distinto de cliente o trabajador), bastará con crear un nuevo tipo que herede del tipo "persona".
- Por último, otra gran ventaja sería que nos permiten crear tablas donde las columnas pueden ser objetos o no. Es decir, si consideramos en nuestro SI como objetos los "Ingrediente", en la tabla de objetos de tipo "Plato" podríamos tener atributos como el nombre, precio... pero también considerar un atributo que fuera una lista de objetos de tipo "Ingrediente". Al incluir una lista de "Ingredientes", cuando se accede a los atributos de un "Plato", se podría también acceder a los atributos de los objetos "Ingrediente" de su lista.

También, debido a esta posibilidad de organización, las consultas se pueden hacer de manera más natural.

Por tanto, considero que el modelo Objeto-Relacional es adecuado para implementar nuestro sistema de información de un restaurante.

2. Modelo NoSQL Documental (Elena Abreu Fernández)

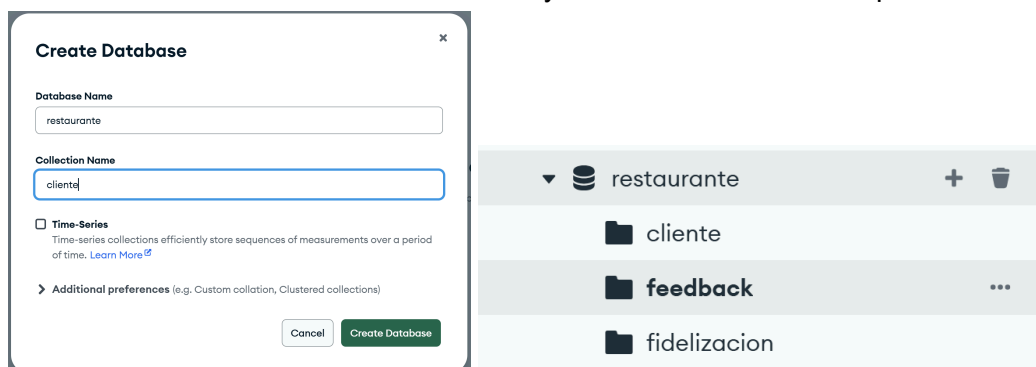
2.1 Instalación

Para el desarrollo de este modelo, he optado por usar MongoDB. Para ahorrar instalaciones y poder usar la interfaz del SGBD he decidido crear una cuenta en MongoDB Atlas, pues como el alcance de nuestro proyecto no va a ser muy extenso, nos sirve su versión gratuita. Para la conexión a la BD hemos usado MongoDB Compass, para su uso solo hemos tenido que descargar el correspondiente .exe y usar el siguiente comando al lanzar la aplicación: `mongodb+srv://elena:<db_password>@ddsi4.nrosify.mongodb.net/?appName=DDSI4`

2.2. DDL y DML utilizado

Para implementar el subsistema de gestión de clientes se ha utilizado un SGBD NoSQL documental (MongoDB), donde la información se almacena en documentos tipo JSON dentro de colecciones. Se ha creado la base de datos restaurante y, dentro de ella, tres colecciones relacionadas con el módulo de clientes:

- cliente: almacena la ficha del cliente (datos de contacto y campos variables como alergias, preferencias y consentimiento de marketing).
- fidelización: almacena información del programa de puntos y nivel (bronce, plata y oro) asociado a cada cliente.
- feedback: almacena valoraciones y comentarios realizados por los clientes.



Además, se han definido índices para mejorar el rendimiento de las búsquedas y, en el caso necesario, garantizar unicidad:

- En la colección cliente se creó un índice único sobre el campo email, con el objetivo de evitar registros duplicados para el mismo correo. También se añadió un índice sobre teléfono para facilitar búsquedas por ese campo.

- En la colección fidelización se creó un índice único sobre el campo id_cliente_email para asegurar que cada cliente tenga, como máximo, un registro de fidelización.
- En la colección feedback se añadió un índice compuesto sobre id_cliente y fecha, permitiendo filtrar rápidamente por cliente y ordenar por fecha de forma eficiente.

The image displays three screenshots of the MongoDB Compass interface, showing the database structure and index definitions for three collections: 'cliente', 'feedback', and 'fidelizacion'.

Top Screenshot (cliente collection): The left sidebar shows the database 'dds44.nrosify.mongodb.net' with collections 'admin', 'config', 'local', 'restaurante', 'cliente', 'feedback', and 'fidelizacion'. The 'cliente' collection is selected. The main panel shows the index definitions for the 'cliente' collection:

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	4.1 kB	1 (since Wed Dec 31 2025)	UNIQUE	READY
> cliente_email_1_fecha_-1	REGULAR	4.1 kB	0 (since Wed Dec 31 2025)	COMPOUND	READY

Middle Screenshot (feedback collection): The 'feedback' collection is selected. The main panel shows the index definitions for the 'feedback' collection:

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	4.1 kB	1 (since Wed Dec 31 2025)	UNIQUE	READY
> email_1	REGULAR	4.1 kB	0 (since Wed Dec 31 2025)	UNIQUE	READY
> telefono_1	REGULAR	4.1 kB	0 (since Wed Dec 31 2025)		READY

Bottom Screenshot (fidelizacion collection): The 'fidelizacion' collection is selected. The main panel shows the index definitions for the 'fidelizacion' collection:

Name & Definition	Type	Size	Usage	Properties	Status
> _id_	REGULAR	4.1 kB	1 (since Wed Dec 31 2025)	UNIQUE	READY
> id_cliente_email_1	REGULAR	4.1 kB	0 (since Wed Dec 31 2025)	UNIQUE	READY
> puntos_-1	REGULAR	4.1 kB	0 (since Wed Dec 31 2025)		READY

Las operaciones realizadas corresponden al DML equivalente de un sistema relacional:

- Inserción de documentos (equivalente a INSERT), para registrar clientes, puntos de fidelización y feedback.
- Actualización de documentos (equivalente a UPDATE), utilizando operadores como \$set, \$inc y \$addToSet.
- Borrado de documentos (equivalente a DELETE), para eliminar registros de comentarios.
- Consultas (equivalente a SELECT) mediante filtros en Compass y comandos find y sort.
- Agregaciones mediante Aggregation Pipeline, para obtener estadísticas (en nuestro caso, media de valoración por cliente).

2.3. Sentencias empleadas y resultados

INSERT:

Se insertaron datos en las tres tablas para simular datos reales del SI.

ddsi4.nrosify.mongodb.net > restaurante > feedback

Documents 2 Aggregations Schema Indexes 2 Validation

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain Reset Find </> Options ▶

ADD DATA EXPORT DATA UPDATE DELETE 25 1 - 2 of 2

▶

```
_id: ObjectId('6954dfb10de0c42a74837737')
id_cliente: 10
fecha: 2025-12-20T00:00:00.000+00:00
rating: 5
comentario: "Rápido y rico"
```

▶

```
_id: ObjectId('6954dfc40de0c42a74837739')
id_cliente: 10
fecha: 2025-12-28T00:00:00.000+00:00
rating: 4
comentario: "Buen trato, algo de espera"
```

ddsi4.nrosify.mongodb.net > restaurante > fidelizacion

Documents 2 Aggregations Schema Indexes 3 Validation

Type a query: { field: 'value' } or [Generate query](#) ⚡

Explain Reset Find </> Options ▶

ADD DATA EXPORT DATA UPDATE DELETE 25 1 - 2 of 2

▶

```
_id: ObjectId('6954df4b0de0c42a74837732')
id_cliente_email: "mario@correo.es"
nivel: "Bronce"
puntos: 20
```

▶

```
_id: ObjectId('6954df830de0c42a74837734')
id_cliente_email: "ana@correo.es"
nivel: "Plata"
puntos: 120
```

Type a query: { field: 'value' } or [Generate query](#)

Explain

Reset

Find

</>

Options

+ ADD DATA

EXPORT DATA

UPDATE

DELETE

25

1 - 2 of 2

↺

↻

↷

☰

🔍

📄

📊

```

_id: 10
nombre: "Ana López"
email: "ana@correo.es"
telefono: "600111222"
fecha_alta: 2025-12-01T00:00:00.000+00:00
  ▶ alergias: Array (1)
  ▶ preferencias: Object
  ▶ marketing: Object

```

```

_id: 11
nombre: "Mario Pérez"
email: "mario@correo.es"
telefono: null
fecha_alta: 2025-12-05T00:00:00.000+00:00
  ▶ alergias: Array (empty)
  ▶ preferencias: Object
  ▶ marketing: Object

```

UPDATE:

Se actualizó la puntuación de un cliente con \$inc, se insertó una nueva alergia con \$addToSet (esto no duplica valores) y se modificó un comentario con \$set.

```

> db.feedback.updateOne(
  { id_cliente: 10, rating: 4 },
  { $set: { rating: 3, comentario: "Buen trato, pero más espera de lo esperado" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}

```

```

> db["feedback"].find()
< {
  _id: ObjectId('6954dfb10de0c42a74837737'),
  id_cliente: 10,
  fecha: 2025-12-20T00:00:00.000Z,
  rating: 5,
  comentario: 'Rápido y rico'
}
{
  _id: ObjectId('6954dfc40de0c42a74837739'),
  id_cliente: 10,
  fecha: 2025-12-28T00:00:00.000Z,
  rating: 3,
  comentario: 'Buen trato, pero más espera de lo esperado'
}

```

```
< switched to db restaurante
> db.fidelizacion.updateOne(
  { id_cliente_email: "ana@correo.es" },
  { $inc: { puntos: 30 } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
Atlas atlas-f9m9af-shard-0 [primary] restaurante>|
```

```
> db["fidelizacion"].find()
< {
  _id: ObjectId('6954df4b0de0c42a74837732'),
  id_cliente_email: 'mario@correo.es',
  nivel: 'Bronce',
  puntos: 20
}
{
  _id: ObjectId('6954df830de0c42a74837734'),
  id_cliente_email: 'ana@correo.es',
  nivel: 'Plata',
  puntos: 150
}
```

```
> db.cliente.updateOne(
  { _id: 10 },
  { $addToSet: { alergias: "lactosa" } }
)
< {
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```

> db["cliente"].find()
< {
  _id: 10,
  nombre: 'Ana López',
  email: 'ana@correo.es',
  telefono: '600111222',
  fecha_alta: 2025-12-01T00:00:00.000Z,
  alergias: [
    'frutos secos',
    'lactosa'
  ],
  preferencias: {
    vegano: false,
    sinGluten: true,
    mesaFavorita: 4
  },
  marketing: {
    consentimiento: true,
    canal: 'email'
  }
}

```

DELETE :

```

> db.feedback.deleteOne({ id_cliente: 10, rating: 5 })
< {
  acknowledged: true,
  deletedCount: 1
}

```

```

> db["feedback"].find()
< {
  _id: ObjectId('6954dfc40de0c42a74837739'),
  id_cliente: 10,
  fecha: 2025-12-28T00:00:00.000Z,
  rating: 3,
  comentario: 'Buen trato, pero más espera de lo esperado'
}

```

CONSULTAS:

Se realizó una búsqueda de un cliente con preferencias concretas y una ordenación ascendente según los puntos.


```
_id: ObjectId('6954ea860de0c42a74837759')
id_cliente: 10
fecha: 2025-12-30T00:00:00.000+00:00
rating: 5
comentario: "Volveré seguro"
```

```
_id: 10
media: 4
n: 2
```

2.4. Discusión sobre la adecuación para el SI

MongoDB resulta adecuado para el subsistema de gestión de clientes de un restaurante debido a la flexibilidad del modelo documental. En este caso, campos como alergias, preferencias o consentimientos de marketing pueden representarse de forma natural mediante arrays y objetos anidados, sin necesidad de un esquema rígido. Además, la posibilidad de añadir nuevos atributos en el futuro (por ejemplo, preferencias de comunicación, historial de incidencias, etc.) es directa y no requiere migraciones complejas.

El uso de índices permite mejorar el rendimiento de consultas frecuentes y garantizar unicidad en campos críticos como el email. Por otro lado, al separar información en varias colecciones (cliente, fidelización y feedback) la integridad referencial no está garantizada por el SGBD como en un modelo relacional; por tanto, la aplicación debe controlar la consistencia (por ejemplo, evitar registros de fidelización sin cliente asociado).

En conclusión, MongoDB encaja bien para este módulo si se prioriza la flexibilidad y la escalabilidad del sistema, siempre que se diseñen correctamente los índices y se controle la consistencia entre colecciones desde la lógica de la aplicación.

3. Modelo NoSQL Clave-Valor (Elena Gallardo Caro)

3.1 Descarga e instalación del SGBD

Para la realización de esta práctica, y dadas las características del modelo Clave-Valor que prioriza la velocidad y el acceso inmediato, hemos optado por utilizar una instancia de Redis alojada en la nube mediante el servicio oficial de demostración interactiva Redis.io (Try Redis).

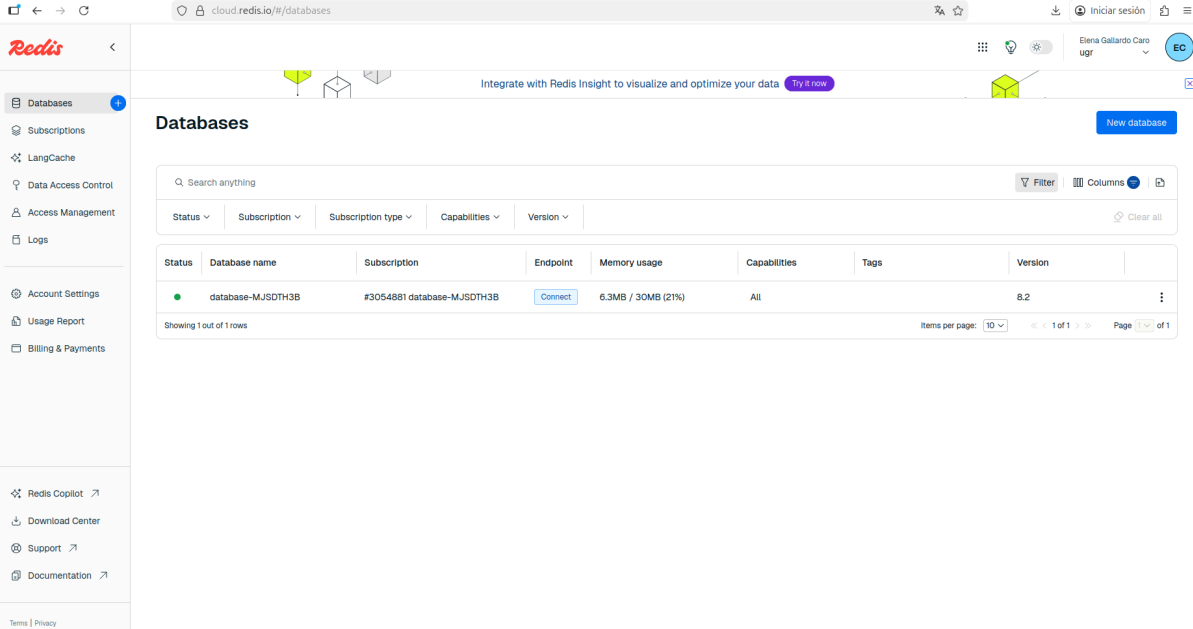
Esta decisión se justifica en la tendencia actual de despliegue de bases de datos NoSQL mediante contenedores (Docker) o servicios gestionados en la nube (DBaaS), evitando instalaciones locales pesadas que dependen del sistema operativo específico.

El proceso de "instalación" o preparación del entorno ha consistido en:

1. Acceso al entorno de ejecución CLI (Command Line Interface) a través del navegador web, que emula un servidor Redis completo.
2. Verificación de la conexión con el servidor mediante el comando PING (que devuelve PONG).
3. Limpieza del espacio de trabajo inicial para asegurar que no hay colisión de claves con sesiones anteriores.

Este entorno proporciona acceso directo al Redis Server, permitiendo la ejecución de sentencias atómicas y la gestión de la memoria volátil en tiempo real, replicando fielmente el comportamiento que tendría el SGBD instalado en un servidor Linux de producción.

Este panel es donde vemos que la base de datos está Status: "Active".



The screenshot shows the Redis.io cloud dashboard. The left sidebar contains navigation links: Databases, Subscriptions, LangCache, Data Access Control, Access Management, Logs, Account Settings, Usage Report, and Billing & Payments. The main content area is titled 'Databases' and features a search bar, filter and column controls, and a table of database instances. The table has columns for Status, Database name, Subscription, Endpoint, Memory usage, Capabilities, Tags, and Version. One instance is listed with a green status dot, indicating it is active.

Status	Database name	Subscription	Endpoint	Memory usage	Capabilities	Tags	Version
●	database-MJSDTH3B	#3054881 database-MJSDTH3B	Connect	6.3MB / 30MB (21%)	All		8.2

Showing 1 out of 1 rows

Items per page: 10 1 of 1 Page 1 of 1

3.2. Descripción del DDL y DML utilizado

A diferencia del modelo Relacional, Redis es una base de datos sin esquema. Esto significa que no existe una fase estricta de DDL (Data Definition Language) donde se creen tablas o se definan columnas antes de insertar datos. La estructura se define dinámicamente en el momento de la inserción. Es decir, que en SQL (Relacional), para ver un pedido tenemos que unir la tabla Clientes, Pedidos y Platos. Aquí NO. Aquí guardamos la información lista para leer.

Por tanto, los comandos utilizados unifican conceptos de definición y manipulación (DML):

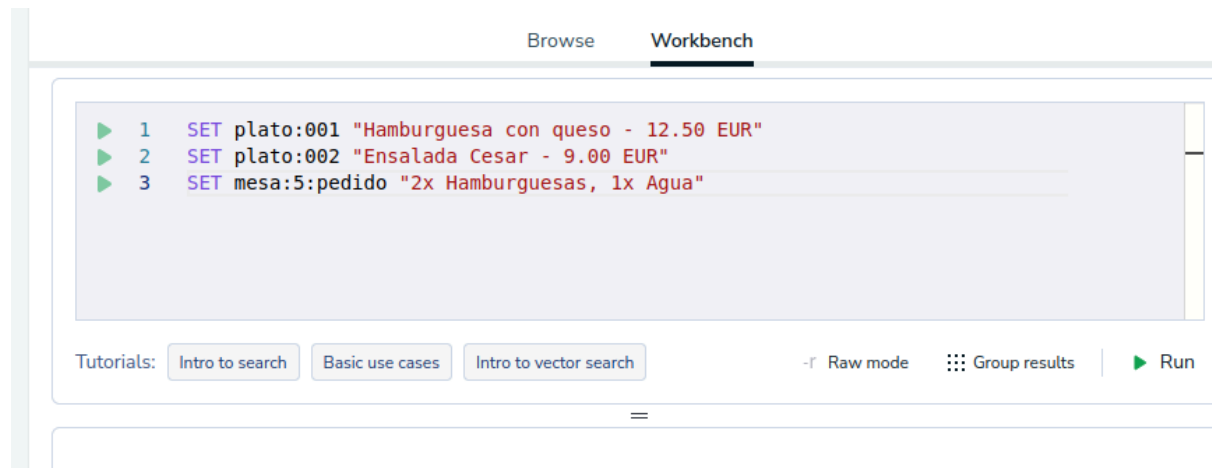
- SET (Creación y Modificación): Es el comando principal. Actúa como una sentencia INSERT si la clave no existe, o como un UPDATE si la clave ya existía. Asocia una cadena de texto (valor) a una etiqueta única (clave).
 - Uso: Se ha utilizado para definir los platos y registrar los pedidos de las mesas.
- GET (Consulta): Equivalente al SELECT en SQL. Recupera el valor almacenado en una clave específica de forma inmediata.
 - Uso: Utilizado para leer el estado actual de un pedido.
- DEL (Borrado): Equivalente al DELETE. Elimina la clave y su valor asociado de la memoria permanentemente.
 - Uso: Para limpiar pedidos finalizados.
- EXISTS (Verificación): Comando lógico para comprobar la existencia de un dato sin necesidad de recuperarlo completo. Útil para evitar errores antes de operaciones críticas.
- EXPIRE (TTL - Time To Live): Una característica distintiva de este modelo. Permite establecer un "tiempo de vida" a un dato, tras el cual el sistema lo borra automáticamente (como un DELETE programado).
 - *Uso:* Fundamental para gestionar reservas temporales.

3.3. Sentencias empleadas y resultados

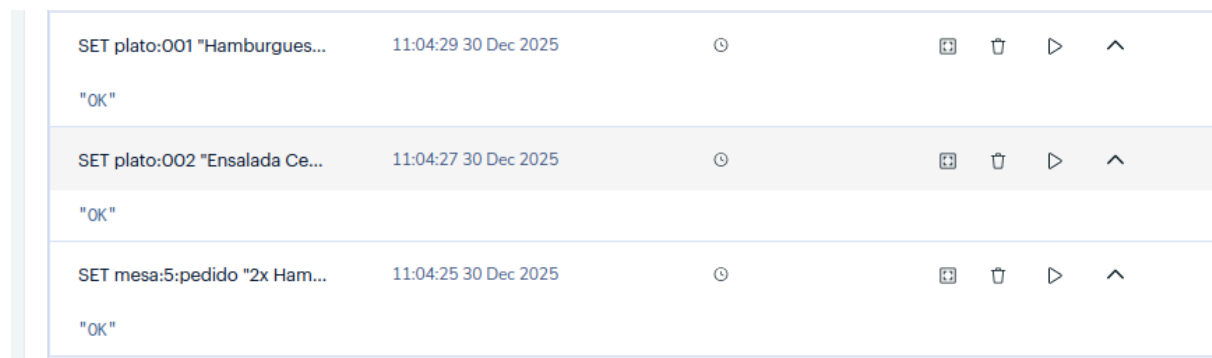
Vamos a simular dos cosas típicas de un restaurante:

1. La carta rápida: Guardar el precio y descripción de un plato para no buscarlo en la base de datos pesada cada vez.
2. Un pedido temporal: Lo que la mesa 5 está pidiendo ahora mismo antes de pagar.

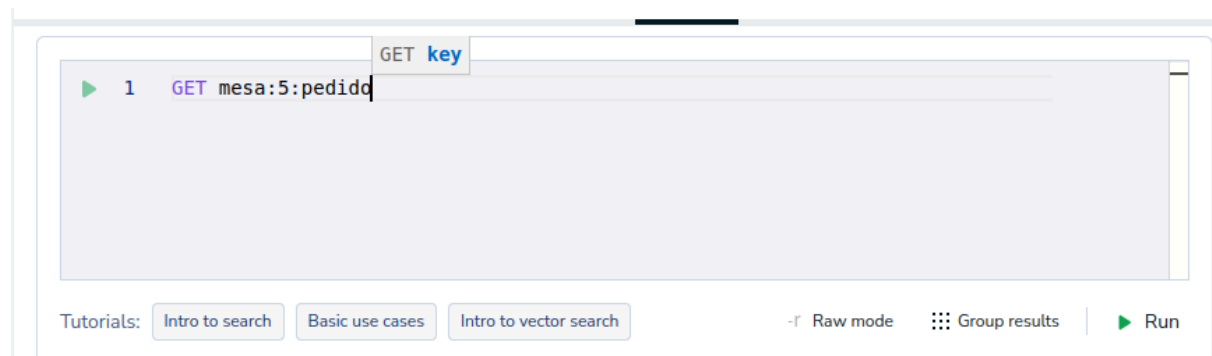
Crear la carta y un pedido (Comando SET): Esto simula que guardas la info de los platos y abres una mesa.



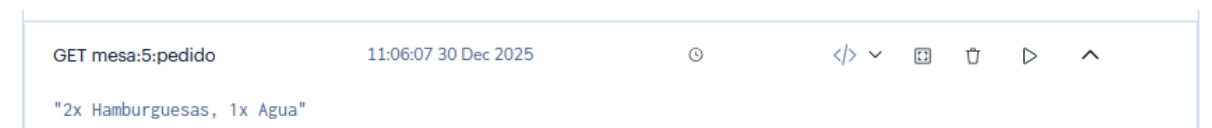
Cuando ejecutamos, nos responde OK si se ha podido realizar, si no nos da error.



Consultar el pedido de la mesa (Comando GET): Esto simula que el camarero mira qué han pedido.



Que nos devuelve:



Con esto además hemos comprobado que es correcta la introducción de antes.

Probar la caducidad (Comando EXPIRE) : Esto simula una reserva que se **cancela** sola si no se confirma. En el simulador online el tiempo pasa muy rápido o a veces no es exacto.

```
1 SET reserva:mesa:10 "Juan Perez (Pendiente)"
2 EXPIRE reserva:mesa:10 20
3
```

Tutorials: [Intro to search](#) [Basic use cases](#) [Intro to vector search](#) -f Raw mode Group results Run

Que nos devuelve:

EXPIRE reserva:mesa:10 20	11:08:13 30 Dec 2025	🕒	📄 🗑️ ▶️ ^
(integer) 1			
SET reserva:mesa:10 "Juan P...	11:08:13 30 Dec 2025	🕒	📄 🗑️ ▶️ ^
"OK"			

Y si esperamos 20 segundos podremos comprobar que el pedido ya se ha cancelado:

GET reserva:mesa:10	11:10:12 30 Dec 2025	🕒	</> 📄 🗑️ ▶️ ^
(nil)			

Y efectivamente, eso es lo que sucede.

Como ya hemos comprobado la Inserción (SET) y Consulta (GET), vamos a hacer una Modificación.

Tenemos ahora:

```
1 SET mesa:5:pedido "2x Hamburguesas, 1x VINO DE LA CASA"
```

Tutorials: [Intro to search](#) [Basic use cases](#) [Intro to vector search](#) -f Raw mode Group results Run

Nos devuelve OK, comprobamos que se ha modificado:

```
1 GET mesa:5:pedido
2
```

Tutorials: [Intro to search](#) [Basic use cases](#) [Intro to vector search](#) -f Raw mode Group results Run

```
GET mesa:5:pedido 11:18:03 30 Dec 2025
"2x Hamburguesas, 1x VINO DE LA CASA"
```

Y todo parece que funciona como debería.

3.4. Discusión sobre la adecuación para el SI

Tras analizar las características de Redis y las necesidades del Sistema de Información (SI) de nuestro restaurante (gestión de clientes, trabajadores, platos, pedidos y reservas), la conclusión es que este modelo NO es adecuado como base de datos única y principal, pero es EXCELENTE como complemento (sistema híbrido).

Argumentos a favor:

- Velocidad en Pedidos en Curso: Un restaurante requiere alta velocidad en cocina y sala. Redis almacena los datos en memoria RAM, lo que permite leer y escribir los pedidos de las mesas activas en milisegundos, mucho más rápido que una base de datos relacional en disco.
- Gestión de "Volatilidad": Muchos datos en un restaurante son temporales. Por ejemplo, el contenido de una comanda antes de cerrarse o una reserva telefónica que espera confirmación. La función EXPIRE de Redis es perfecta para manejar estos datos que no necesitamos guardar para siempre.

Argumentos en contra:

- Consultas Complejas: Redis no permite hacer "Joins" (uniones). Si quisiéramos saber "¿Qué camarero atendió la mesa que pidió la hamburguesa el mes pasado?", sería muy difícil de programar, ya que no podemos cruzar tablas de Trabajadores y Pedidos fácilmente.
- Integridad Referencial: No podemos obligar al sistema a asegurar que un pedido pertenezca a un cliente existente (como hacen las claves foráneas en SQL).

Conclusión: La arquitectura ideal para nuestro restaurante sería Híbrida:

1. Mantendríamos el Modelo Relacional (Oracle/MySQL) para los datos históricos, facturas, contabilidad y listado de empleados (datos que no cambian mucho y requieren seguridad).
2. Utilizaríamos Redis (Clave-Valor) como una capa de "caché" o memoria rápida para gestionar lo que está pasando ahora mismo en el restaurante (mesas abiertas y cocina), volcando los datos al sistema relacional solo cuando se cierra la cuenta y se emite la factura.

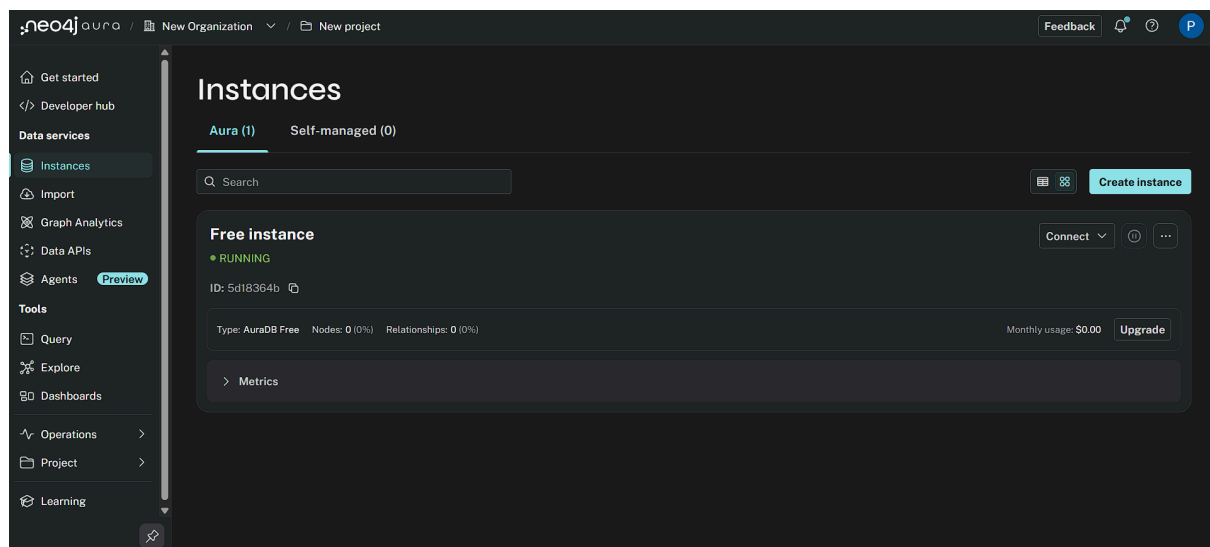
4. Modelo NoSQL Orientado a Grafos (Pablo Bolaños Martínez)

4.1. Descripción de la descarga e instalación del SGBD

Para la realización de esta práctica, he seleccionado el SGBD Neo4j, líder en bases de datos orientadas a grafos. Dado que el proyecto requiere persistencia de datos durante varias semanas para su evaluación y defensa, he descartado la opción de "Sandbox" temporal y he optado por desplegar una instancia en **Neo4j AuraDB Free Tier**.

Esta solución es una base de datos como servicio (DBaaS) en la nube que permite mantener la instancia activa de forma indefinida. Aunque el servicio cuenta con un mecanismo de "pausa automática" tras 72 horas de inactividad para liberar recursos, garantiza la persistencia de los datos y permite reactivar la instancia instantáneamente desde la consola de administración sin pérdida de información.

Para la interacción con la base de datos, utilizo la interfaz web integrada **Neo4j Browser**, conectada de forma segura a la instancia en la nube mediante credenciales generadas durante el aprovisionamiento. Como se muestra en la captura, la instancia se encuentra activa ("Running") y asegurada.



4.2. Descripción del DDL y DML utilizado

A diferencia del modelo relacional que usa SQL, Neo4j utiliza un lenguaje de consulta declarativo propio llamado **Cypher**. Este lenguaje está diseñado para ser intuitivo y legible, representando visualmente los patrones de nodos y relaciones mediante arte ASCII (paréntesis para nodos, flechas para relaciones).

Los comandos básicos utilizados son:

- **CREATE:** Equivalente al INSERT de SQL. Se usa para crear nodos (entidades) y relaciones (conexiones entre entidades).
- **MATCH:** Equivalente al SELECT de SQL. Se usa para buscar patrones en el grafo.
- **RETURN:** Especifica qué datos devolver tras un MATCH.
- **SET:** Equivalente al UPDATE. Se usa para añadir o modificar propiedades en nodos o relaciones existentes.
- **DELETE / DETACH DELETE:** Equivalente al DELETE. Se usa para borrar nodos y relaciones (DETACH se usa para desconectar relaciones antes de borrar el nodo).

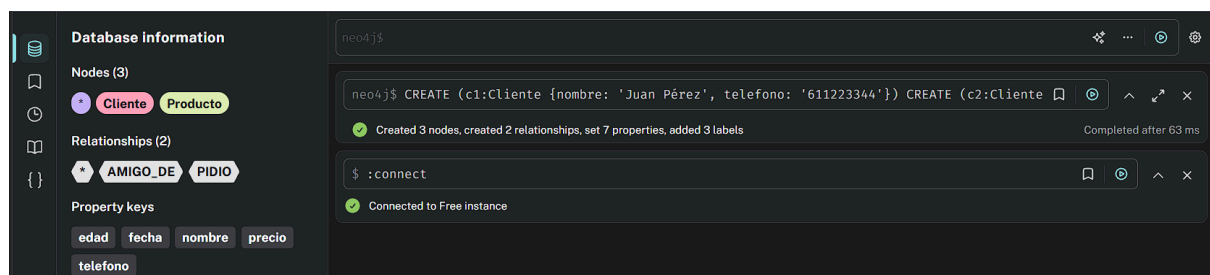
4.3. Sentencias empleadas y Resultados

Para adecuar el ejemplo al Sistema de Información del restaurante, hemos modelado una estructura donde los **Cientes** no solo realizan pedidos de **Productos**, sino que mantienen relaciones sociales entre ellos, algo difícil de representar eficientemente en el modelo relacional.

(A) Creación de Nodos y Relaciones (Inserción): Insertamos dos clientes ("Juan" y "Ana") y un producto estrella ("Solomillo"). Establecemos que Juan pidió el plato y que además es amigo de Ana.

Código ejecutado:

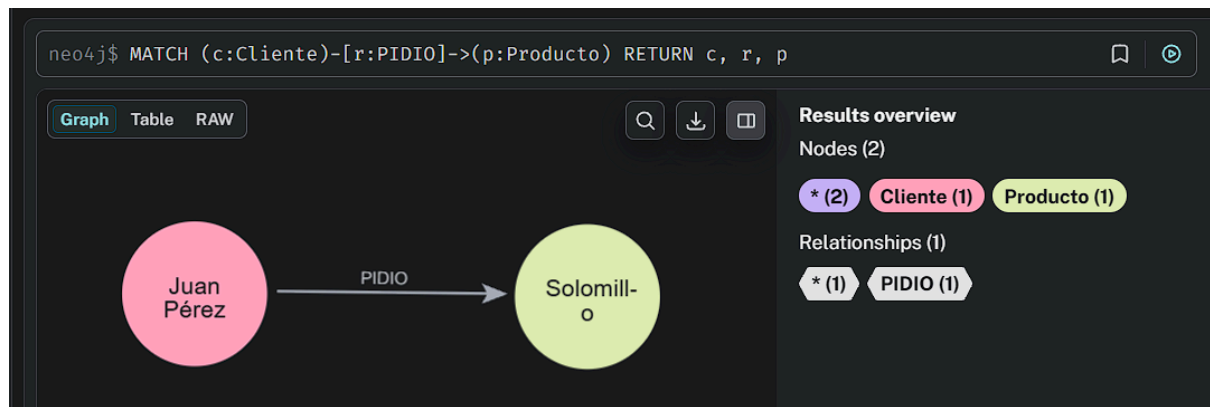
```
CREATE (c1:Cliente {nombre: 'Juan Pérez', telefono: '611223344'})
CREATE (c2:Cliente {nombre: 'Ana García', telefono: '699887766'})
CREATE (p:Producto {nombre: 'Solomillo', precio: 15.50})
CREATE (c1) -[:AMIGO_DE] -> (c2)
CREATE (c1) -[:PIDIO {fecha: '2025-01-29'}] -> (p)
```



(B) Consultas (Lectura): Consultamos quién ha pedido el producto y visualizamos el grafo resultante.

Código ejecutado:

```
MATCH (c:Cliente) -[r:PIDIO] -> (p:Producto)
RETURN c, r, p
```



Node details

Cliente

Key	Value
<id>	4:d2bc9e90-0385-47b8-92b c-3c5a6216afd2:6
nombre	"Juan Pérez"
telefono	"611223344"

Node details

Producto

Key	Value
<id>	4:d2bc9e90-0385-47b8-92b c-3c5a6216afd2:8
nombre	"Solomillo"
precio	15.5

Relationship details

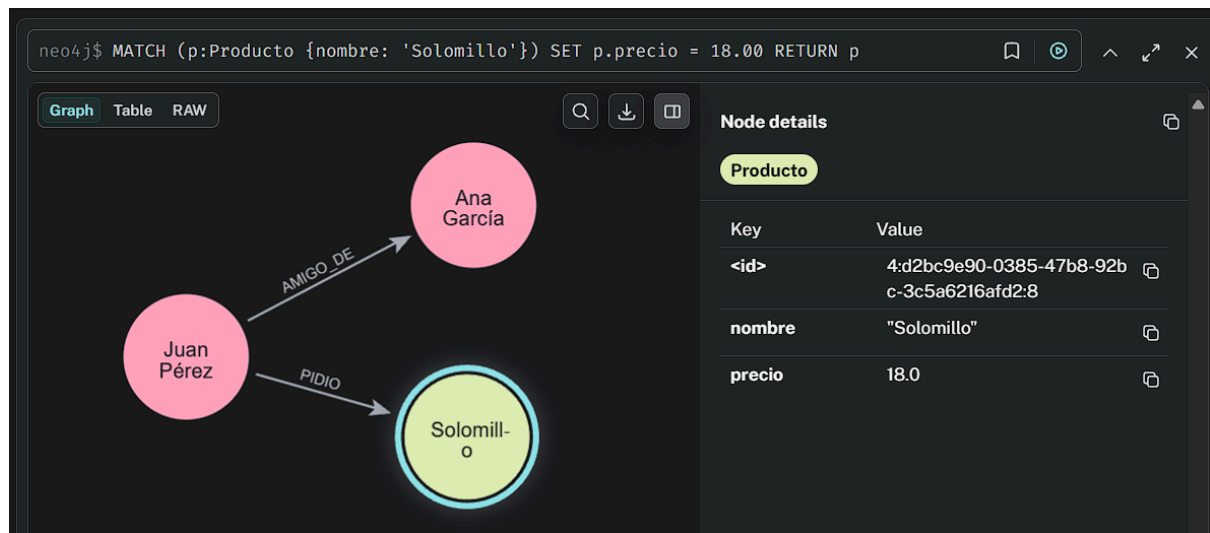
PIDIO

Key	Value
<id>	5:d2bc9e90-0385-47b8-92b c-3c5a6216afd2:1152923703 630102534
fecha	"2025-01-29"

(C) Modificación de Datos: Subimos el precio del plato a 18€, demostrando la flexibilidad del esquema para modificar propiedades sin alterar la estructura del grafo.

Código ejecutado:

```
MATCH (p:Producto {nombre: 'Solomillo'})
SET p.precio = 18.00
RETURN p
```



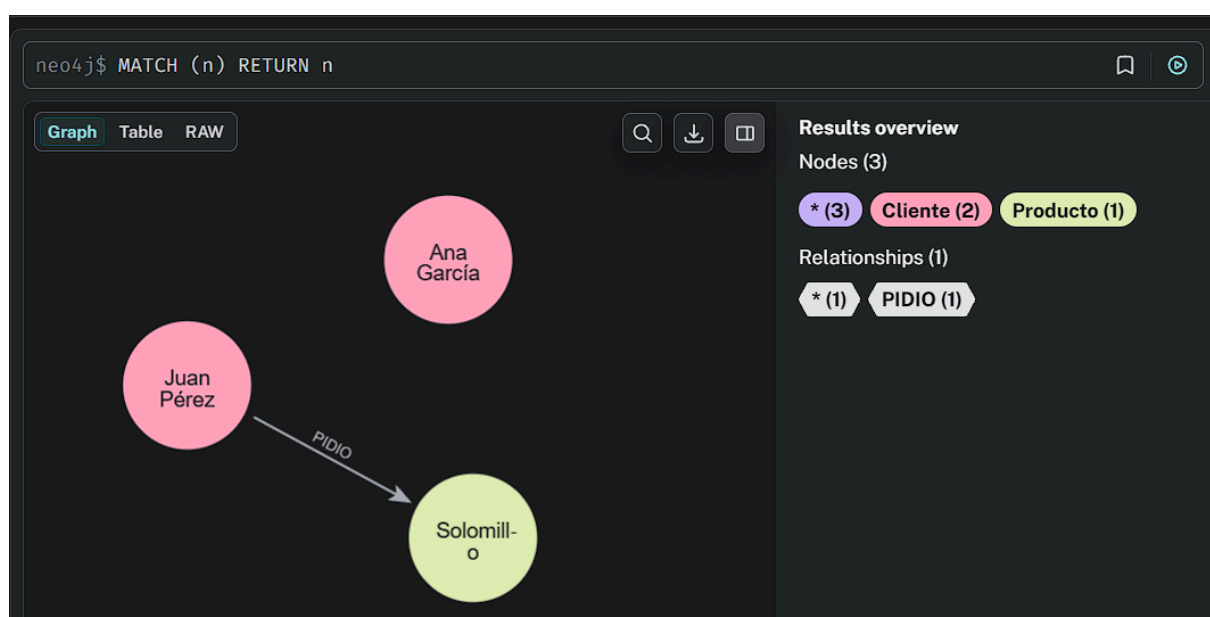
(D) Borrado de Datos: Eliminamos la relación de amistad entre clientes.

Código ejecutado:

```
MATCH (c1:Cliente {nombre: 'Juan Pérez'}) -[r:AMIGO_DE]->(c2:Cliente)
DELETE r
```

```
neo4j$ MATCH (c1:Cliente {nombre: 'Juan Pérez'}) -[r:AMIGO_DE]->(c2:Cliente) DELETE r
Deleted 1 relationship
```

Como podemos observar, se ha eliminado la relación de amistad entre Juan Pérez y Ana García sin necesidad de borrar ningún usuario.



4.4. Discusión sobre la adecuación para el SI

Para el funcionamiento central de nuestra práctica (gestión de platos, gestión de pedidos y reservas, y gestión de clientes), el uso exclusivo de una base de datos orientada a grafos como **Neo4j no sería la opción más eficiente**. La naturaleza de nuestros datos operativos es muy estructurada y tabular, por lo que el modelo Relacional (SQL) que hemos usado en la asignatura resulta más natural y sencillo para operaciones simples de listado y suma.

Sin embargo, **sí sería muy adecuado utilizar Neo4j como sistema complementario** para potenciar el negocio con funciones que el modelo relacional no resuelve bien: **el marketing relacional**.

En nuestra prueba de concepto hemos demostrado que, al conectar Clientes y Productos mediante relaciones (*:AMIGO_DE*, *:PIDIO*), podemos descubrir información valiosa que en una tabla SQL pasaría desapercibida. Por ejemplo, podríamos implementar un sistema de **"Recomendaciones Inteligentes"** en la app del restaurante, sugiriendo a un cliente probar *"el plato favorito de sus amigos"* o *"productos que suelen pedir otros clientes con gustos similares"*. Para este fin específico, Neo4j es muy superior y mucho más rápido que SQL.

5. Modelo Orientado a Objetos (Leandro Jorge Fernández Vega)

5.1 Características

Para la realización del trabajo, se ha optado por el SGBD ZODB (Zope Object Database), que permite almacenar objetos Python de forma persistente sin necesidad de transformarlos a estructuras relacionales. A diferencia de los SGBD tradicionales, ZODB elimina la separación entre el modelo de datos y el modelo de programación, permitiendo trabajar directamente con objetos persistentes. Este paradigma incluye todas las posibilidades de la programación orientada a objetos: herencia, funciones, objetos, etc.

ZODB dispone de un único punto de entrada persistente denominado *root*. Este objeto actúa como contenedor principal desde el que se accede a todo el grafo de objetos persistentes. En la práctica, contiene diccionarios persistentes que representan colecciones de objetos.

No existen comprobaciones para claves primarias o externas. Es el programador el que debe definir estas comprobaciones.

5.2 Instalación

Su instalación es sencilla. Podemos usar el sistema operativo *Linux* y el editor *VSCo*de. Tras la creación de un entorno virtual clásico para proyectos de python, instalamos la biblioteca: `$ pip install zodb`

5.3 Conexión a la base de datos y cierre

El almacenamiento físico se realiza mediante *FileStorage*, que genera un fichero *.fs* y varios ficheros auxiliares (*.index*, *.lock*).

```
import datetime
import ZODB, ZODB.FileStorage
import DDL
import DML
import queries

def main():
    # Abrir conexión ZODB y obtener root
    storage = ZODB.FileStorage.FileStorage('../db/restaurante.fs')
    db = ZODB.DB(storage)
    connection = db.open()
    root = connection.root()
```

```
finally:
    try:
        connection.close()
        db.close()
        storage.close()
        print ("Conexión cerrada correctamente.\n")
    except Exception:
        pass
```

5.4. Descripción del DDL y DML utilizado

Este paradigma trabaja directamente con objetos y clases, sin necesidad de estructuras relacionales. Usaremos funciones pero llevar a cabo cada labor.

- **CREATE:** Se define una clase por cada entidad.

```
from persistent import Persistent
import transaction

class Persona(Persistent):
    def __init__(self, dni, nombre, apellidos, telefono, email):
        self.dni = dni
        self.nombre = nombre
        self.apellidos = apellidos
        self.telefono = telefono
        self.email = email

class Cliente(Persona): # Herencia
    def __init__(self, dni, nombre, apellidos, telefono, email, contrasena, estado_sesion="Desconectado"):
        super().__init__(dni, nombre, apellidos, telefono, email)
        self.contrasena = contrasena
        self.estado_sesion = estado_sesion

class Trabajador(Persona): # Herencia
    def __init__(self, dni, nombre, apellidos, telefono, email, cargo, estado="Activo"):
        super().__init__(dni, nombre, apellidos, telefono, email)
        self.cargo = cargo
        self.estado = estado

class Reserva(Persistent): # Objeto Complejo
    def __init__(self, id_reserva, cliente, trabajador, num_personas, fecha_hora, estado="Pendiente"):
        self.id_reserva = id_reserva
        self.cliente = cliente # Referencia directa al objeto Cliente
        self.trabajador = trabajador # Referencia directa al objeto Trabajador
        self.num_personas = num_personas
        self.fecha_hora = fecha_hora
        self.estado = estado

def create_tables(root):
    if 'clientes' not in root:
        root['clientes'] = {}
    if 'trabajadores' not in root:
        root['trabajadores'] = {}
    if 'reservas' not in root:
        root['reservas'] = {}
    root._p_changed = True
    transaction.commit()
```

- **DROP:**

```
def drop_tables(root):
    for key in ['clientes', 'trabajadores', 'reservas']:
        if key in root:
            del root[key]
            root._p_changed = True
    transaction.commit()
```

- INSERT:

```
import transaction
from DDL import Cliente, Trabajador, Reserva
import datetime
```

```
def crear_cliente(root, cliente: Cliente):
    clientes = root.get(['clientes', {}])
    if cliente.dni in clientes:
        raise KeyError(f"Cliente {cliente.dni} ya existe")
    clientes[cliente.dni] = cliente
```

```
def insertar_datos_ejemplo(root):
    # Crear objetos (DML - Inserción)
    cli = Cliente("12345678A", "Leandro", "Fernández", "600111222", "lea@email.com", "1234")
    tra = Trabajador("87654321B", "Ana", "López", "600333444", "ana@rest.com", "Cocinero")

    # Persistir objetos con comprobación de clave primaria
    try:
        crear_cliente(root, cli)
    except KeyError as e:
        print(f"{e}\n")
```

- DELETE:

```
def delete_reserva(root, id_reserva: str):
    reservas = root.get('reservas', {})
    if id_reserva not in reservas:
        raise KeyError(f"Reserva {id_reserva} no existe")

    del reservas[id_reserva]
    transaction.commit()
```

- UPDATE:

```
def update_reserva_reasignar_trabajador(root, id_reserva: str, dni_trabajador: str):
    reservas = root.get('reservas', {})
    trabajadores = root.get('trabajadores', {})
    if id_reserva not in reservas:
        raise KeyError(f"Reserva {id_reserva} no existe")
    if dni_trabajador not in trabajadores:
        raise KeyError(f"Trabajador {dni_trabajador} no existe")

    res = reservas[id_reserva]
    res.trabajador = trabajadores[dni_trabajador] # referencia directa a objeto
    res._p_changed = True
    transaction.commit()
    return res
```

- CONSULTAS:

```
def listar_reservas_detalladas(root):
    print("--- LISTADO DE RESERVAS ---\n")
    reservas = root.get('reservas', {})

    for id_res, res in reservas.items():
        # Navegación natural entre objetos (Sin necesidad de JOINS)
        print(f"Reserva ID: {id_res}\n")
        print(f"  Cliente: {res.cliente.nombre} {res.cliente.apellidos}\n")
        print(f"  Gestionada por: {res.trabajador.nombre} ({res.trabajador.cargo})\n")
        print(f"  Fecha/Hor (variable) res: Any ")
        print(f"  Estado: {res.estado}\n")
        print(("-" * 30) + "\n")

def reservas_por_cliente(root, dni_cliente: str):
    reservas = root.get('reservas', {})
    return [r for r in reservas.values() if r.cliente.dni == dni_cliente]

def conteo_reservas_por_dia(root):
    reservas = root.get('reservas', {})
    counts = {}
    for r in reservas.values():
        d = r.fecha_hora.date()
        counts[d] = counts.get(d, 0) + 1
    return counts
```

--- LISTADO DE RESERVAS ---

Reserva ID: RES-001

Cliente: Leandro Fernández

Gestionada por: Ana (Cocinero)

Fecha/Hora: 2025-12-31 21:00:00

Estado: Confirmada

Reservas del cliente 12345678A: [<DDL.Reserva object at 0x7a59195fb3f0>]

Conteo reservas por día: {datetime.date(2025, 12, 31): 1}

5.5. Discusión sobre la adecuación para el SI

El modelo orientado a objetos resulta conceptualmente muy adecuado para la gestión de un restaurante, ya que permite representar de forma natural las entidades del dominio, como clientes, trabajadores, reservas, etc., así como las relaciones directas entre ellas mediante referencias. Este enfoque facilita una correspondencia clara entre el modelo de datos y el modelo de programación, reduciendo la complejidad del desarrollo.

No obstante, en un entorno real con necesidades de consulta intensiva, análisis de datos, generación de informes y posible integración con otros sistemas corporativos (facturación, contabilidad o plataformas externas), el modelo orientado a objetos presenta ciertas limitaciones. La ausencia de un lenguaje de consulta estandarizado, la gestión manual de la integridad de los datos y las dificultades para realizar agregaciones complejas o consultas globales afectan a la escalabilidad y a la explotación eficiente de la información.

Por este motivo, aunque el modelo orientado a objetos resulta adecuado desde el punto de vista del modelado conceptual y especialmente útil en prototipos o sistemas de tamaño reducido, no suele ser la opción más apropiada para un sistema de gestión de restaurante en producción. En estos casos, el modelo relacional ofrece mayores garantías en términos de estandarización, optimización de consultas, control de la integridad y soporte por parte de herramientas y tecnologías ampliamente consolidadas en el ámbito empresarial.